

Software Testing

Software testing is a critical quality assurance activity in the software development lifecycle. It is not merely the act of finding bugs; it is a systematic investigation conducted to provide stakeholders with information about the quality of the software product.

1. Introduction to Software Testing

Software testing is the process of evaluating a system or its components with the intent to find whether it satisfies the specified requirements and to identify any defects. The IEEE Standard 610.12-1990 defines testing as *"the process of operating a system or component under specified conditions, observing or recording the results, and making an evaluation of some aspect of the system or component."*

A broader definition encompasses both static activities (reviews, walkthroughs, inspections) and dynamic activities (executing the code). The primary goal is to uncover errors, gaps, and missing requirements as early as possible, thereby reducing the risk of software failure and improving overall product quality. Testing is a destructive process in the sense that a good test is one that has a high probability of finding a yet-undiscovered defect.

2. Why Testing Code is Important?

The significance of testing extends far beyond simple error detection:

1. **Economic Rationale:** The cost of fixing a defect grows exponentially the later it is found. A logic error discovered during requirements analysis costs almost nothing; the same error found after deployment can cause immense financial and reputational damage.
 2. **Product Quality and Reliability:** In safety-critical systems (avionics, medical devices, autonomous vehicles), a software failure can result in loss of life. Rigorous testing provides a measure of confidence in the software's reliability.
 3. **Customer Satisfaction and Trust:** Users abandon unreliable applications. Thorough testing ensures a positive user experience, which is essential for business reputation and customer retention.
 4. **Security:** With increasing cyber threats, testing for vulnerabilities (injection flaws, broken authentication, sensitive data exposure) is non-negotiable. A single security gap can lead to data breaches and legal consequences.
 5. **Maintainability:** Systems with comprehensive test suites can be refactored and enhanced with confidence. The test suite acts as a safety net, catching regressions introduced by code changes—a cornerstone of DevOps and continuous delivery.
-

3. Verification and Validation

These two terms are often used interchangeably but represent distinct concepts critical to software quality.

- **Verification:** *"Are we building the product right?"* It is the process of evaluating the intermediate work products (requirements, design documents, code) to ensure they conform to the specifications and standards established for that phase. Verification is a static activity; it does not involve executing the code. Examples include reviews, walkthroughs, and inspections.

- **Validation:** *"Are we building the right product?"* It is the process of evaluating the final product to ensure it meets the business needs and the user's expectations. Validation is a dynamic activity; it involves executing the software under real or simulated conditions. Examples include system testing, user acceptance testing, and beta testing.

Verification and validation are complementary. Verification aims to prevent defects, while validation aims to detect failures. The V-Model (discussed later) explicitly aligns verification activities with each development phase and validation activities with each corresponding testing phase.

4. Quality Assurance vs. Quality Control vs. Testing

These three terms form a hierarchy of quality management.

Term	Scope	Focus	Orientation	Examples
Quality Assurance (QA)	Organization-wide, process-oriented.	Establishing and maintaining a defined set of processes and standards to ensure quality is built into the product from the start.	Preventive. Aims to avoid defects by improving the process.	Process definition, audits, training, selection of development methodologies.
Quality Control (QC)	Product-oriented, a subset of QA.	Verifying that the product meets the defined quality standards. Detecting defects in the finished product.	Detective. Aims to identify and correct defects before release.	Reviews, inspections, testing.
Testing	A core activity within QC.	The dynamic execution of software to find defects and evaluate quality attributes.	Detective. It is one of the most common QC techniques.	Unit testing, integration testing, system testing, automated regression suites.

The relationship is hierarchical: **QA encompasses QC, and QC encompasses Testing**. An organization with a strong QA culture has processes that make it easy to produce high-quality code, while QC (including testing) acts as the check to confirm those processes are working.

5. Principles of Software Testing

The International Software Testing Qualifications Board (ISTQB) defines seven fundamental principles that guide all testing efforts:

1. **Testing shows the presence of defects, not their absence.** Testing can prove that defects exist, but it can never prove that software is defect-free. It reduces the probability of undiscovered defects but provides no guarantee of perfection.
2. **Exhaustive testing is impossible.** Testing all possible combinations of inputs and preconditions is infeasible for any non-trivial system. Instead, risk-based prioritization, equivalence partitioning, and boundary value analysis are used to optimize test coverage.

3. **Early testing saves time and money.** Testing activities should start as early as the requirements and design phases (static testing) to detect defects when they are cheap to fix. This aligns with the "shift-left" philosophy.
 4. **Defect clustering.** A small number of modules usually contain the majority of the defects (Pareto principle: 80% of defects are found in 20% of the code). Testing effort should be concentrated on these complex, error-prone areas after they are identified.
 5. **Pesticide paradox.** If the same set of tests are run repeatedly, they will eventually stop finding new defects. To overcome this, test cases must be regularly reviewed, revised, and new, more creative test scenarios must be introduced.
 6. **Testing is context dependent.** Testing a safety-critical avionics system is fundamentally different from testing an e-commerce mobile app. The methodology, techniques, and rigor must be tailored to the nature, risk, and usage context of the software.
 7. **Absence-of-errors fallacy.** Finding and fixing defects does not help if the system built is unusable or does not fulfill the user's needs and expectations. A perfectly bug-free system that no user wants is a failure. Validation is as important as verification.
-

6. Introduction to STLC and V-Model

6.1 Software Testing Life Cycle (STLC)

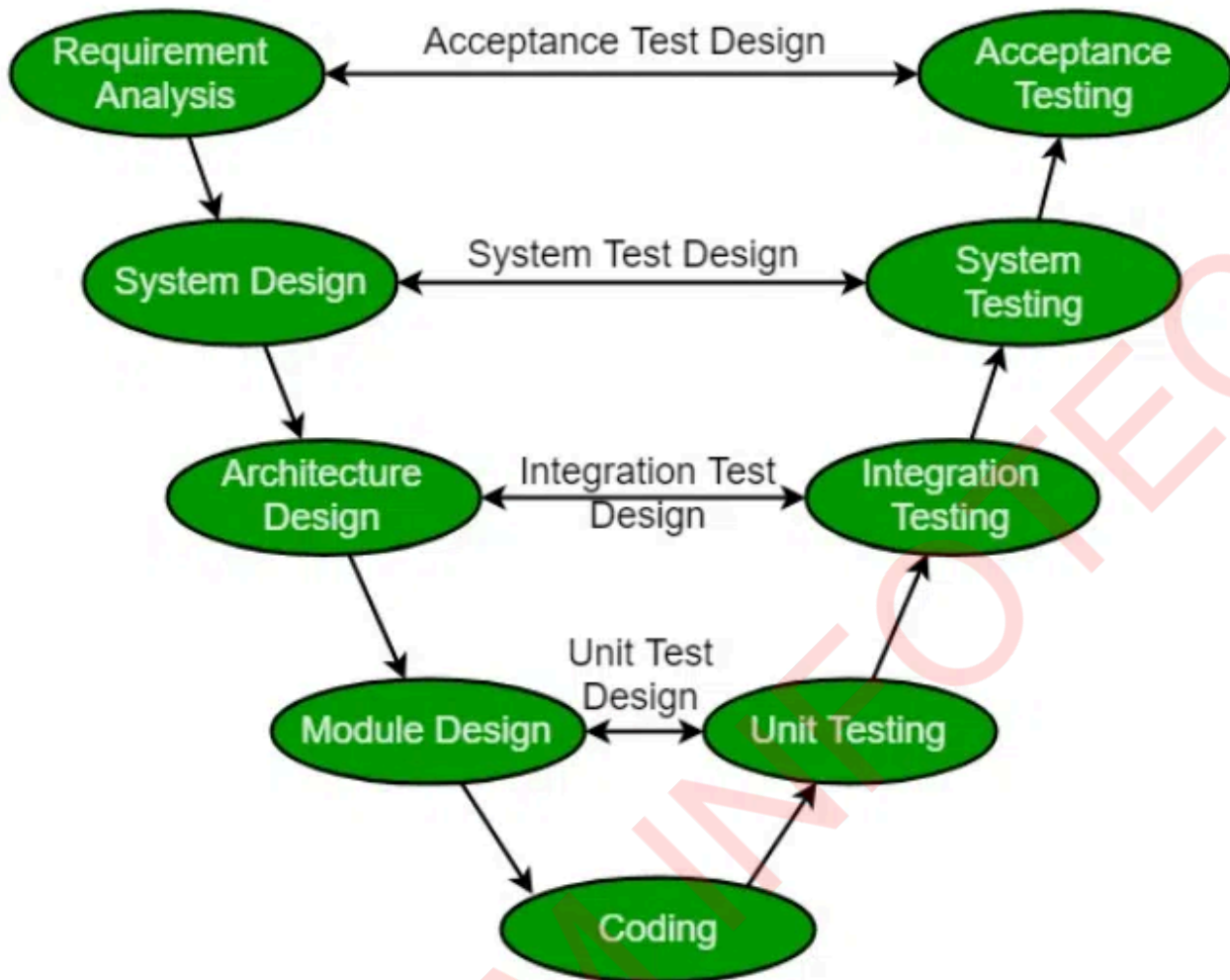
The STLC is a sequence of specific activities conducted during the testing process to ensure software quality goals are met. It is a part of the broader SDLC but focused exclusively on the testing phase.

The phases of STLC are:

1. **Requirement Analysis:** The testing team studies the requirements (SRS, use cases) to identify testable requirements. Ambiguities are clarified, and a Requirement Traceability Matrix (RTM) is initiated to map requirements to test cases.
2. **Test Planning:** A comprehensive Test Plan document is created. It defines the scope, objectives, strategy, resources, schedule, entry/exit criteria, and risks of the testing effort.
3. **Test Case Development:** Based on the test plan and requirements, detailed test cases are written. Each case includes steps, input data, expected results, and preconditions. Test data is prepared, and the test environment requirements are specified.
4. **Test Environment Setup:** The hardware, software, network, and test data configurations are established to replicate the production environment as closely as possible. Smoke testing is performed to verify the environment's readiness.
5. **Test Execution:** The test cases are executed. Actual results are recorded and compared against expected results. Defects are logged in a defect tracking tool, and status is tracked against the plan. Regression testing is performed after defect fixes.
6. **Test Cycle Closure:** This final phase involves evaluating the cycle completion criteria (e.g., all planned tests executed, defect threshold met). A Test Summary Report is prepared, lessons learned are documented, and test artifacts are archived for future use.

6.2 The V-Model (Verification and Validation Model)

The V-Model is an SDLC model where execution happens in a sequential manner in a V-shape. It emphasizes the symbiotic relationship between each development phase and its corresponding testing phase. The left side of the 'V' represents the verification (static) phases, and the right side represents the validation (dynamic) phases.



- **Unit Testing:** Validates the smallest testable parts (modules, functions, classes) against their detailed design specification.
- **Integration Testing:** Verifies that multiple units interact correctly as per the architectural design. Tests interfaces and interactions.
- **System Testing:** Validates the fully integrated system against the system design and functional/non-functional requirements. It is a black-box test of the complete product.
- **Acceptance Testing:** Validates the system against business requirements. It ensures the software meets the end-user's needs and is ready for deployment. This includes User Acceptance Testing (UAT), Alpha, and Beta tests.

The V-Model's strength is its discipline and the early link between testing and development phases. However, like Waterfall, it is rigid and not ideal for projects with rapidly changing requirements.

7. Types of Testing: Manual and Automation

Testing can be performed in two broad modes based on the involvement of human testers.

Aspect	Manual Testing	Automation Testing
Execution	Test cases are executed step-by-step by a human tester without the assistance of scripts or tools.	Test scripts are written and executed using automation tools.
Best Suited For	Exploratory, usability, and ad-hoc testing; test cases that run once or twice; scenarios requiring human intuition and observation.	Regression testing; repetitive, data-driven tests; performance and load testing; high-risk, high-frequency test cases.
Initial Investment	Low for tools; high for human effort over time.	High initial cost for tool purchase/license and script development; low cost for repeated execution.
Accuracy	Prone to human error, especially with repetitive and complex checks.	Consistent and precise in checking predefined conditions; no fatigue.
Coverage	Limited by the number of testers and available time.	Can run hundreds of tests on multiple platforms simultaneously, improving breadth.
Maintenance	Test cases may become outdated; minimal change effort.	Scripts require maintenance when the application UI or logic changes, which can be a significant effort.

A balanced testing strategy combines both: manual testing for new features and exploratory work, and automation for stable regression suites, building towards a CI/CD pipeline.

8. Tools Used for Automation Testing

A wide ecosystem of tools supports automated testing across different layers and domains. Common categories and examples include:

- **Unit Testing Frameworks (Developers):** JUnit, TestNG (Java); Jest, Mocha (JavaScript); unittest, pytest (Python). They integrate with build tools and CI servers.
- **Functional/E2E Web Testing:** Selenium WebDriver (supports multiple languages, including Java, Python, JS, etc), Cypress, Playwright. These drive browsers to simulate user actions.
- **API Testing:** Postman, REST Assured (Java), SoapUI. Validate backend service contracts without a UI.
- **Performance Testing:** JMeter, Gatling, Locust. Simulate heavy loads to measure response times and throughput.
- **Mobile Testing:** Appium (cross-platform), Espresso (Android), XCUITest (iOS).
- **CI/CD Integration:** Jenkins, GitHub Actions, GitLab CI/CD trigger automated test suites on every commit.

The choice of tool depends on the application stack, team skill set, and testing objectives. For a Java-based backend, JUnit with REST Assured is a natural fit; for a MERN frontend, Jest and Cypress are common.

9. Introduction to Testing Methods: White-Box, Black-Box, and Grey-Box

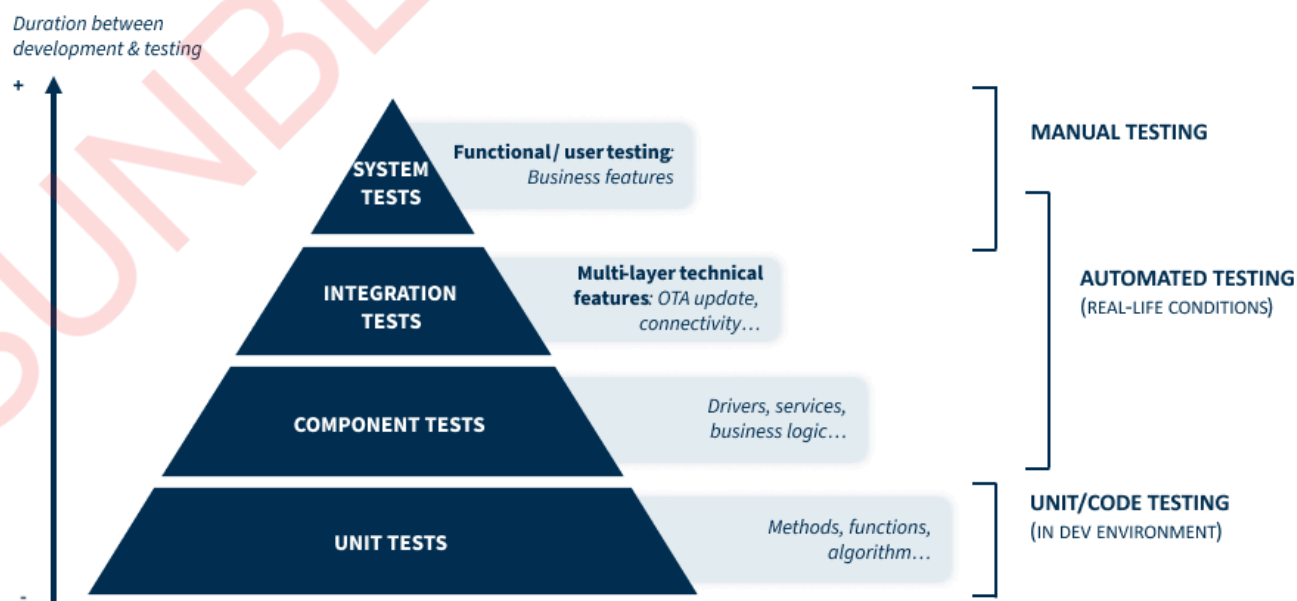
Testing methods describe the perspective from which test cases are derived, based on the tester's knowledge of the internal structure.

Method	Internal Knowledge	Focus	Techniques	Typical Phases
White-Box Testing	Full knowledge of internal code, logic, and structure. Code is visible.	Verifying the internal logic, control flow, and data flow. Statements, branches, and paths.	Statement coverage, branch/decision coverage, path coverage, loop testing.	Unit testing, integration testing (conducted by developers).
Black-Box Testing	No knowledge of internal code. Only inputs and outputs are known.	Validating functionality against specifications and requirements. The system is a "black box."	Equivalence partitioning, boundary value analysis, decision tables, state transition testing, use case testing.	System testing, acceptance testing.
Grey-Box Testing	Partial knowledge of internal design and architecture (e.g., database schema, API contracts, algorithms).	Tests both functionality and structural aspects, aiming to find context-specific errors like broken links or improper session management.	Matrix testing, regression testing, pattern testing, orthogonal array testing.	Integration testing, security testing, business domain testing.

Grey-box testing is particularly powerful for web applications where a tester knows the HTTP request/response contracts and the database schema, allowing them to design tests that expose data integrity and security flaws.

10. Testing Pyramid

The **Testing Pyramid** is a conceptual model that describes the ideal distribution of automated tests across different testing levels. It emphasizes creating a large number of fast, reliable, low-cost tests at the lower levels of the system and a smaller number of slower, expensive tests at the higher levels. The model helps teams achieve faster feedback, lower maintenance effort, and more stable continuous integration pipelines.



Unit Testing (Base Layer)

Unit tests validate the smallest testable parts of an application, such as methods, functions, or classes, in isolation from external dependencies.

- **Purpose:** Verify internal business logic and algorithm correctness.
- **Characteristics:** Fast execution, low maintenance cost, highly stable.
- **Execution Frequency:** Run frequently during development and on every build.
- **Examples:** Testing a Java method that calculates tax or validates login credentials.
- **Common Tools:** JUnit, TestNG, Jest, pytest.

Because unit tests are inexpensive and provide immediate feedback, they form the largest layer of the pyramid.

Integration Testing (Middle Layer)

Integration tests verify that multiple modules or services interact correctly with each other.

- **Purpose:** Validate interfaces, API contracts, database communication, and service integration.
- **Characteristics:** Slower than unit tests, require more setup and environment configuration.
- **Examples:** Testing whether a REST API correctly stores and retrieves data from a database.
- **Common Tools:** REST Assured, Postman, Spring Test.

Integration tests provide confidence that independently working components function correctly when combined.

Note: Component (e.g. Service) testing can be considered a subset of integration testing.

UI / End-to-End (E2E) Testing (Top Layer)

End-to-End tests validate complete user workflows from the perspective of a real user interacting with the system.

- **Purpose:** Ensure the entire application behaves correctly in real-world scenarios.
- **Characteristics:** Slowest and most expensive tests; more fragile due to UI and environment dependencies.
- **Examples:** Login → Search Product → Add to Cart → Checkout → Payment Confirmation.
- **Common Tools:** Selenium WebDriver, Playwright, Cypress.

Only the most critical business workflows should be automated at the E2E level to avoid excessive execution time and maintenance effort.

Importance of the Testing Pyramid

The Testing Pyramid promotes an efficient testing strategy by balancing speed, reliability, and coverage.

Testing Level	Speed	Cost	Stability	Recommended Quantity
Unit Tests	Very Fast	Low	High	High
Integration Tests	Moderate	Medium	Moderate	Medium
UI/E2E Tests	Slow	High	Lower	Low

A well-structured testing pyramid helps teams:

- Detect defects early.
- Reduce debugging effort.
- Improve CI/CD pipeline efficiency.
- Minimize flaky and unstable tests.
- Maintain high software quality with faster feedback cycles.

An over-reliance on UI automation creates an “ice-cream cone anti-pattern,” where excessive top-level tests become difficult to maintain and slow down development. Therefore, modern testing strategies emphasize strong unit and integration test coverage with a smaller set of critical E2E tests.

11. Introduction to Functional Testing

Functional testing is a type of testing that validates the software system against the functional requirements/specifications. It focuses on *what* the system does. The goal is to verify each function of the application by feeding appropriate input and comparing the output with the expected behavior.

Functional testing answers questions like:

- Does the login work with valid credentials?
- Is an error message displayed for invalid inputs?
- Does adding an item to the cart update the cart total?

Key functional testing levels (ordered by the phase of execution in the V-Model):

Unit Testing

- **Scope:** The smallest testable part of an application—a function, method, or class.
- **Performed By:** Developers, during the coding phase.
- **Characteristics:** White-box; automated using frameworks like JUnit (Java). A single unit is tested in isolation; dependencies are mocked/stubbed. Foundation of the test pyramid.

Integration Testing

- **Scope:** Interaction between two or more integrated units or modules.
- **Performed By:** Developers or dedicated integration testers.
- **Characteristics:** Can be grey-box. Approaches include Big-Bang (all modules together), Top-Down, Bottom-Up, and Sandwich. Validates interface contracts, data communication, and error handling between modules (e.g., testing the interaction between a Node.js controller and the MySQL database layer).

Smoke Testing (Build Verification Testing)

- **Scope:** A shallow but wide set of tests that check the core functionality of a new build.
- **Purpose:** To determine if the build is stable enough to warrant more rigorous and time-consuming testing. It is the “gatekeeper” for accepting a build into the testing environment.

System Testing

- **Scope:** The entire, fully integrated software system.
- **Performed By:** An independent testing team in an environment mimicking production.

- **Characteristics:** Black-box. Validates end-to-end workflows against the SRS and system design. Both functional and non-functional testing are performed at this level.

Acceptance Testing

- **Scope:** The complete system from the user's perspective, verifying business requirements.
- **Performed By:** End-users, clients, or product owners.
- **Alpha Testing:** Conducted at the developer's site by the internal team or a controlled group of users. The environment is controlled, and developers can observe and log issues quickly.
- **Beta Testing:** Conducted at the client's site or in a real-world environment by a limited set of external users who are not part of the development team. Feedback on usability, compatibility, and real-world bugs is collected before a public release.
- **User Acceptance Testing (UAT):** The final formal test to confirm the system meets contractual and business requirements, often leading to sign-off for deployment.

Regression Testing

- **Scope:** Executing a suite of existing test cases to ensure that recent code changes (new features, bug fixes) have not adversely affected existing functionality.
- **Purpose:** To catch unintended side-effects. Regression suites are prime candidates for automation because they run frequently.

12. Introduction to Non-Functional Testing

Non-functional testing verifies the *how* aspects of the system—the attributes that define the user's experience, not a specific behavior. It is critical for ensuring the system is usable, fast, and secure.

Performance Testing

Evaluates system responsiveness, stability, and resource usage under a specific workload. Sub-types include:

- **Load Testing:** Assesses behavior under expected normal and peak load conditions.
- **Stress Testing:** Pushes the system beyond its normal operational capacity, often to the breaking point, to evaluate robustness and error handling under extreme conditions (e.g., simulating 10x the expected user traffic). This directly addresses the "Stress testing" topic.
- **Endurance (Soak) Testing:** Applies a significant load over an extended period to detect memory leaks and performance degradation.
- **Spike Testing:** Tests the system's reaction to sudden, large spikes in load.

Security Testing

Identifies vulnerabilities, threats, and risks in the software to prevent malicious attacks. It aims to ensure confidentiality, integrity, authentication, authorization, and availability. Techniques include vulnerability scanning, penetration testing, and SQL injection/XSS testing—especially vital for typical web applications handling payments and personal data.

Usability Testing

Evaluates the user-friendliness, intuitiveness, and overall experience of the application. It involves observing real users as they attempt to complete tasks, measuring factors like learnability, efficiency of use, error rate, and subjective satisfaction. This is inherently a manual, human-centric process.

Portability Testing

Checks whether the software can run correctly across different environments such as operating systems, browsers, devices, hardware configurations, or databases with minimal changes. It ensures the application is adaptable, installable, and compatible across platforms. Examples include testing a web application on Windows, Linux, Android, iOS, Chrome, Firefox, and other browsers.

13. Practical Artifact: Create a Test Plan for an E-Commerce Project

A Test Plan is a strategic document that defines the scope, approach, resources, and schedule of intended testing activities. Below is a structured outline with content tailored for a hypothetical "MERN E-Commerce Platform" project.

TEST PLAN Project: MERN E-Commerce Platform (MEC) **Version:** 1.0 **Date:** [Date]

1. Introduction

- **Purpose:** This document describes the test strategy for the MEC web application. It ensures that the system meets functional and non-functional requirements as defined in SRS v2.1.
- **Scope:** Testing will cover the core modules: User Authentication, Product Catalog & Search, Shopping Cart, Checkout & Payment Gateway Integration (Stripe sandbox), Order Management, and Admin Dashboard.

2. Test Items

- Frontend: React application (v18)
- Backend: Node.js/Express REST APIs
- Database: MySQL 8.0 with Sequelize ORM
- Integration touchpoints: Payment gateway, Email service (SMTP)

3. Features to be Tested

- User registration, login, password reset (functional, security).
- Product listing, filtering, sorting, and search (functional, performance).
- Add to cart, update quantity, remove item (functional, integration).
- Complete checkout flow with mock payment (end-to-end, security).
- Admin CRUD operations on products and orders (functional).

4. Features Not to be Tested

- Backend batch processing for vendor payouts (not in scope for v1.0).
- Mobile responsive behavior below 320px width (low priority, deferred).

5. Testing Strategy

- **Levels & Types:** Unit testing (developers, Jest/JUnit for any microservices), Integration testing (API tests with Postman), System testing (manual exploratory and automated E2E with Cypress), UAT (product owner).
- **Methods:** Combination of black-box (user-facing scenarios) and grey-box (API-level validation).

- **Non-functional:** Load test with JMeter for catalog and checkout APIs (1000 concurrent users), OWASP ZAP scan for top 10 security risks, and usability walkthrough with 5 representative users.
- **Automation:** API tests will run in CI on every push; critical user journeys (login, purchase) automated with Cypress.

6. Entry and Exit Criteria

- **Entry:** All unit tests passing on the feature branch; static code analysis clean; build deployed to the staging environment successfully; smoke test passed.
- **Exit:** All critical/high severity defects fixed; test coverage targets met (e.g., 85% functional test case pass rate); no open high-priority bugs; UAT signed off.

7. Resources and Environment

- **Hardware:** Cloud VMs (AWS EC2 t3.medium) running Linux.
- **Software:** Node.js, React, MySQL, Git, Jenkins CI, Jira for defect tracking.
- **Team:** 2 QA Engineers, 1 Product Owner, all developers for unit tests.

8. Schedule and Risks

- **Milestones:** Test planning (Day 1-2), Case development (Day 3-7), Environment setup (Day 8), Execution cycles (Day 9-18 per sprint).
- **Risks:** Delay in third-party sandbox availability (mitigation: use mocked stubs early), incomplete requirements (mitigation: close collaboration with PO, exploratory testing).

9. Deliverables

- Test Plan (this document), RTM, Test Cases, Defect Reports, Test Summary Report.

14. Practical Artifact: Create a Test Case Document for Different Sprints

A test case is a set of conditions and steps to validate a specific feature. For an Agile project, test cases evolve with each sprint. Below is a sample Test Case Document format with entries from two different sprints.

Test Case Document Template

Field	Description
Test Case ID	Unique identifier, e.g., TC-LOGIN-001
Test Scenario	Brief description of the high-level scenario being tested.
Sprint	The sprint in which the test case is designed/executed.
Pre-conditions	State of the system before test execution (e.g., user exists, database state).
Test Steps	Numbered, unambiguous steps to execute.
Test Data	Specific inputs, files, or credentials.
Expected Result	The precise system behavior after each step.
Actual Result	Filled during execution; Pass/Fail, and notes.
Status	Pass, Fail, Blocked, Skipped.

Sample Test Cases: E-Commerce Project

Sprint 1: User Authentication Module

Test Case ID: TC-LOGIN-001 **Scenario:** Verify successful login with valid credentials. **Pre-conditions:** User account `john@example.com` with password `Pass@123` is registered and active. User is on the login page. **Steps:**

1. Enter email `john@example.com` in the email field.
2. Enter password `Pass@123` in the password field.
3. Click "Sign In". **Test Data:** Email: `john@example.com`, Password: `Pass@123`. **Expected Result:** User is redirected to the home page; a JWT token is stored in local storage; a success toast message "Login successful" is displayed.

Test Case ID: TC-LOGIN-002 **Scenario:** Verify error message for invalid password. **Pre-conditions:** Same user exists. User on login page. **Steps:**

1. Enter email `john@example.com`.
2. Enter incorrect password `WrongPass`.
3. Click "Sign In". **Test Data:** Email: `john@example.com`, Password: `WrongPass`. **Expected Result:** System remains on login page; error message "Invalid email or password" is displayed; no token is set.

Sprint 2: Product Catalog & Search

Test Case ID: TC-SEARCH-001 **Scenario:** Verify search returns matching products by name. **Pre-conditions:** Database contains products: "Wireless Mouse", "Gaming Keyboard", "USB-C Hub". User is on the product listing page. **Steps:**

1. Click on the search bar.
2. Type "mouse" into the field.
3. Press Enter or click the search icon. **Test Data:** Search string: `mouse`. **Expected Result:** The product list reloads showing only "Wireless Mouse". A breadcrumb or label "Results for 'mouse'" is displayed. "Gaming Keyboard" and "USB-C Hub" are not visible.

Test Case ID: TC-CART-001 **Scenario:** Verify "Add to Cart" from search results page. **Pre-conditions:** User is logged in; a product "Wireless Mouse" is in the search results. **Steps:**

1. Click the "Add to Cart" button next to "Wireless Mouse".
2. Navigate to the cart page. **Test Data:** Product ID 101 (Wireless Mouse). **Expected Result:** Cart icon in header updates to show "1". Cart page displays the product, quantity 1, price, and a total that matches the product price.

NOTE: These test case documents are living artifacts; they are updated when requirements change and are linked to the RTM for full traceability. In a Jira-based Scrum project, each test case can be a sub-task under a user story, ensuring that "Done" means all linked test cases have passed.

Application Build

Application build process and tools depends on the development stack. You can learn them in respective modules. As here we are discussing about testing Java applications and UI applications using Java Selenium, you need to be aware of the Java build tools. (Though not part of this module).

Java Build tools: Maven, Gradle, Ant.

Maven Introduction:

- <https://youtu.be/IMXBrIVFYA0?si=rfdiqwhR8btFiINf>
- <https://jenkov.com/tutorials/maven/maven-tutorial.html>

Automated Testing with JUnit and Selenium

Now let's bridge the gap between testing theory and hands-on automation, directly supporting the modern DevOps and Agile practices discussed earlier. Here we will learn the essentials of the JUnit 5 unit testing framework for Java and Selenium WebDriver for browser-based UI automation.

1. Introduction

Automated testing is the technical engine behind continuous integration and rapid feedback. While manual testing has its place (exploratory, usability), any test that must run frequently—especially regression, smoke, and end-to-end flows—should be automated.

- **JUnit** is the de-facto standard for unit testing in Java. It validates the smallest testable parts of your application (methods, classes) in isolation.
- **Selenium WebDriver** automates real browsers, enabling you to test the complete UI layer exactly as a user would interact with it.

2. JUnit 5 Essentials

2.1 What is JUnit?

JUnit 5 is the latest generation of the Java unit testing framework. It consists of three sub-projects:

- **JUnit Platform:** Foundation for launching testing frameworks on the JVM.
- **JUnit Jupiter:** New programming model and extension model for writing tests.
- **JUnit Vintage:** Provides backward compatibility for JUnit 3 and 4 tests.

For all new projects, Jupiter is used exclusively.

2.2 Core Annotations and Test Lifecycle

JUnit uses annotations to mark methods that control the test flow. Understanding the lifecycle is critical.

Annotation	Purpose	Lifecycle Scope
@Test	Denotes a test method.	—
@BeforeAll	Method runs once before all test methods. Must be static .	Per class
@AfterAll	Method runs once after all test methods. Must be static .	Per class
@BeforeEach	Method runs before each test method.	Per method
@AfterEach	Method runs after each test method.	Per method
@Disabled	Skips a test (with an optional reason).	—

Annotation	Purpose	Lifecycle Scope
<code>@DisplayName</code>	Provides a human-readable name for the test.	—
<code>@Nested</code>	Groups related tests in an inner class, allowing shared setup.	—

Example – Lifecycle Demo:

```
import org.junit.jupiter.api.*;

public class LifecycleTest {

    @BeforeAll
    static void initAll() {
        System.out.println("Before all tests - DB connection pool init");
    }

    @AfterAll
    static void tearDownAll() {
        System.out.println("After all tests - Cleanup resources");
    }

    @BeforeEach
    void init() {
        System.out.println(" Before each test - Create fresh test data");
    }

    @AfterEach
    void tearDown() {
        System.out.println(" After each test - Reset state");
    }

    @Test
    @DisplayName("Test addition with positive numbers")
    void testAddition() {
        System.out.println(" Running addition test");
        // test logic
    }

    @Test
    @Disabled("Feature not yet implemented")
    void testSubtraction() {
        // skipped
    }
}
```

2.3 Assertions

Assertions are the core of a test; they verify expected outcomes. JUnit Jupiter provides a rich set of static assertion methods in `org.junit.jupiter.api.Assertions`.

```
import static org.junit.jupiter.api.Assertions.*;

@Test
void standardAssertions() {
    assertEquals(4, 2 + 2, "Optional failure message");
    assertNotEquals(5, 2 + 2);
    assertTrue(1 > 0);
    assertFalse(0 > 1);
    assertNull(null);
    assertNotNull(new Object());
}

@Test
void testException() {
    // Verify that the executable throws ArithmeticException
    Exception exception = assertThrows(ArithmeticException.class, () -> {
        int result = 1 / 0;
    });
    assertEquals("/ by zero", exception.getMessage());
}

@Test
void groupedAssertions() {
    // All assertions are evaluated and failures are reported together
    assertAll("person",
        () -> assertEquals("John", person.getFirstName()),
        () -> assertEquals("Doe", person.getLastName())
    );
}
```

2.4 A Complete Unit Test Example

Consider a simple `Calculator` class:

```
public class Calculator {
    public int add(int a, int b) { return a + b; }
    public int divide(int a, int b) { return a / b; }
}
```

Corresponding JUnit 5 test:

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {

    private Calculator calc;

    @BeforeEach
```

```
void setUp() {
    calc = new Calculator(); // fresh instance before each test
}

@Test
@DisplayName("Test add method with positive numbers")
void testAdd() {
    assertEquals(5, calc.add(2, 3), "2 + 3 should equal 5");
}

@Test
@DisplayName("Test divide by zero throws exception")
void testDivideByZero() {
    assertThrows(ArithmeticException.class, () -> calc.divide(10, 0));
}
}
```

Running this in Eclipse (right-click → Run As → JUnit Test) displays a green bar for passes and red for failures.

2.5 Execute Test Cases with Maven

Maven automates the execution of your entire test suite with a single command, making it the bridge between local development and continuous integration pipelines.

The Maven Surefire Plugin

The Surefire plugin is responsible for running the unit tests of an application. It is bound to the `test` phase of the default Maven lifecycle. Surefire automatically detects JUnit 5 if the required Jupiter dependencies are on the classpath and the provider `junit-platform-surefire-provider` is available (this provider is now embedded in the `maven-surefire-plugin` itself). To be explicit and avoid version conflicts, it is best practice to include the Surefire plugin in your `pom.xml` with a clear version.

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
    </plugin>
  </plugins>
</build>
```

With this configuration, Surefire will scan `src/test/java` for classes whose names match any of the default patterns:

`**/Test*.java, **/*Test.java, **/*Tests.java, **/*TestCase.java.`

Executing All Tests

Open a terminal at the root of your Maven project and run:

```
mvn clean test
```

- `clean` removes previous build outputs to ensure a fresh start.
- `test` triggers the default lifecycle up to the `test` phase: compile source, compile test source, execute tests.

During execution, you will see output like:

```
[INFO] --- maven-surefire-plugin:3.2.5:test (default-test) @ your-artifact-id ---
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.CalculatorTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.05 s
[INFO] Running com.example.LoginUITest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.20 s
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
```

All test classes in `src/test/java` are executed. The Selenium UI tests are included because they follow the naming convention and reside in the same test source folder.

Test Reports

After execution, Maven generates plain-text and XML reports in the `target/surefire-reports` directory. These reports contain details for each test method, including execution time, status, and stack traces for failures. The XML format is compatible with CI tools like Jenkins, which can parse them to publish graphical test trend reports.

To browse a quick summary, open:

```
target/surefire-reports/*.txt
```

Separating Unit and Integration Tests (Optional)

It is common to classify Selenium UI tests as "integration tests" because they require external resources (a browser, a running server). Maven provides the **Failsafe Plugin** for this purpose. Tests in `src/test/java` that follow naming patterns like `**/IT*.java`, `**/*IT.java`, `**/*ITCase.java` are executed by Failsafe during the `integration-test` phase, not the `test` phase.

To adopt this separation:

1. Move your Selenium test classes to `src/test/java/.../LoginUITest.java` and rename them to `LoginIT.java`.
2. Add the Failsafe plugin:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-failsafe-plugin</artifactId>
  <version>3.2.5</version>
</plugin>
```

3. Run integration tests separately:

```
mvn verify
```

The **verify** phase runs after **integration-test** and ensures both Surefire unit tests and Failsafe integration tests are executed. Your CI pipeline would typically execute **mvn clean verify** to obtain a complete picture of code quality.

3. Selenium WebDriver Essentials

3.1 What is Selenium?

Selenium is an open-source umbrella project for browser automation. The most widely used component, **Selenium WebDriver**, provides a programming interface to drive browser behavior natively, as a real user would. It supports multiple languages; here we use Java.

3.2 Project Setup in Eclipse

Step 1: Create a Maven project in Eclipse. **Step 2:** Add dependencies to **pom.xml** for Selenium and WebDriverManager:

```
<dependencies>
  <!-- Selenium Java (WebDriver) -->
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.44.0</version> <!-- Use latest stable -->
  </dependency>

  <!-- JUnit 5 -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.1</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Step 3: Create a Java class in **src/test/java** to write tests.

3.3 Loading the WebDriver

WebDriverManager automatically downloads and configures the correct driver binary for your browser.

Starting from Selenium 4.6.0, Selenium includes an in-built tool called Selenium Manager that automatically handles the download, caching, and configuration of browser drivers (such as ChromeDriver) when you instantiate a WebDriver object (e.g., new ChromeDriver()).

- Automatic Management: Selenium Manager detects the installed browser version and downloads the compatible driver executable automatically.
- Simplified Code: You can simply use `WebDriver driver = new ChromeDriver();` without any prior setup calls.

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class SetupDriver {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();
        driver.get("https://www.example.com");
        System.out.println("Title: " + driver.getTitle());
        driver.quit();
    }
}
```

For production tests, always set a reasonable **implicit wait** so the driver polls the DOM for a certain duration before throwing a `NoSuchElementException`:

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

3.4 Selenium Commands: Locators

Locators are the mechanism to identify HTML elements on a page. WebDriver provides `findElement()` and `findElements()` with eight `By` strategies.

Locator	Description	Example
<code>By.id</code>	Unique id attribute (fastest)	<code>driver.findElement(By.id("login-btn"));</code>
<code>By.name</code>	name attribute	<code>driver.findElement(By.name("email"));</code>
<code>By.className</code>	CSS class name (avoid compound classes)	<code>driver.findElement(By.className("btn-primary"));</code>

Locator	Description	Example
<code>By.tagName</code>	HTML tag name	<code>driver.findElement(By.tagName("h1"));</code>
<code>By.xpath</code>	XPath expression (very flexible)	<code>driver.findElement(By.xpath("//input[@name='password']"));</code>
<code>By.cssSelector</code>	CSS selector (often faster than XPath)	<code>driver.findElement(By.cssSelector("#username"));</code>
<code>By.linkText</code>	Exact text of a link	<code>driver.findElement(By.linkText("Forgot Password?"));</code>
<code>By.partialLinkText</code>	Partial link text	<code>driver.findElement(By.partialLinkText("Forgot"));</code>

XPath Details

XPath is the most powerful locator when elements lack stable IDs. Basic syntax:

- `//tag[@attribute='value']` – element with exact attribute match.
- `//tag[contains(@attribute, 'partial-value')]` – partial match.
- `//tag[text()='exact text']` – exact text content.
- Axes: `//div[@id='main']//child::span` or shortcuts like `../` (parent), `/following-sibling::*`.

Example:

```
// Login button with dynamic class containing 'submit'
driver.findElement(By.xpath("//button[contains(@class, 'submit-btn')]"));
// Label following a specific input field
driver.findElement(By.xpath("//label[text()='Password']/following-sibling::input"));
```

3.5 Interactions: Basic HTML Controls

Once an element is located, you can interact with it using the `WebElement` interface.

```
WebElement emailField = driver.findElement(By.id("email"));
emailField.clear(); // Clear existing text
emailField.sendKeys("user@example.com"); // Type text

WebElement loginButton = driver.findElement(By.id("submit"));
loginButton.click(); // Click the button

String headingText = driver.findElement(By.tagName("h1")).getText();
String attributeValue = emailField.getAttribute("placeholder");
```

```
// Dropdown (select element) handling
WebElement dropdown = driver.findElement(By.id("country"));
Select select = new Select(dropdown);
select.selectByVisibleText("India");
select.selectByValue("IN");
```

3.6 Keyboard and Mouse Actions

For complex user interactions, Selenium provides the **Actions** class. It supports chaining of events like hover, drag-and-drop, double-click, and modifier keys.

```
Actions actions = new Actions(driver);

// Mouse hover (move to element)
WebElement menu = driver.findElement(By.id("products-menu"));
actions.moveToElement(menu).perform();

// Double-click
WebElement btn = driver.findElement(By.id("double-click-me"));
actions.doubleClick(btn).perform();

// Drag and drop
WebElement source = driver.findElement(By.id("draggable"));
WebElement target = driver.findElement(By.id("droppable"));
actions.dragAndDrop(source, target).perform();

// Keyboard actions (e.g., Ctrl+A, Backspace)
WebElement input = driver.findElement(By.id("search-box"));
actions.click(input)
    .keyDown(Keys.CONTROL)
    .sendKeys("a")
    .keyUp(Keys.CONTROL)
    .sendKeys(Keys.BACK_SPACE)
    .perform();

// Complex example: Hover, then click a sub-menu
actions.moveToElement(menu)
    .moveToElement(subMenu)
    .click()
    .build()
    .perform();
```

Always end the action chain with `.perform()`. For chains that should be built once and reused, use `.build()` then `.perform()`.

4. Selenium Use Cases

Automated browser control opens up three practical domains: UI testing, web scraping, and task automation.

4.1 Web Scraping

Extracting data from web pages is straightforward when you locate elements and read text/attributes.

```
WebDriver driver = new ChromeDriver();
driver.get("https://books.toscrape.com");

// Extract all book titles from the page
List<WebElement> bookTitles = driver.findElements(By.xpath("//h3/a"));
for (WebElement book : bookTitles) {
    System.out.println(book.getAttribute("title"));
}
driver.quit();
```

4.2 Task Automation

Automating repetitive web forms, such as submitting a daily report, saves time and reduces human error.

```
// Automate a contact form submission
driver.get("https://example.com/contact");
driver.findElement(By.id("name")).sendKeys("John Doe");
driver.findElement(By.id("email")).sendKeys("john@example.com");
new Select(driver.findElement(By.id("department"))).selectByVisibleText("Sales");
driver.findElement(By.id("message")).sendKeys("Automated daily check-in.");
driver.findElement(By.id("submit")).click();
// Verify success message
String successMsg = driver.findElement(By.cssSelector(".alert-success")).getText();
System.out.println("Success: " + successMsg);
```

4.3 UI Testing

Combining Selenium with JUnit yields powerful, maintainable UI tests. The following is a complete login test, following best practices.

```
public class LoginUITest {

    private WebDriver driver;

    @BeforeEach
    void setUp() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(5));
    }

    @AfterEach
    void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

```
}

@Test
@DisplayName("Login with valid credentials")
void testValidLogin() {
    driver.get("https://mystore.example.com/login");

    // Enter credentials
    driver.findElement(By.id("email")).sendKeys("customer@test.com");
    driver.findElement(By.id("password")).sendKeys("SecurePass123!");
    driver.findElement(By.id("login-button")).click();

    // Assert presence of a welcome element
    WebElement welcomeMsg = driver.findElement(By.cssSelector(".welcome-
banner"));
    assertTrue(welcomeMsg.getText().contains("Welcome back"));
}

@Test
@DisplayName("Login with invalid password shows error")
void testInvalidLogin() {
    driver.get("https://mystore.example.com/login");
    driver.findElement(By.id("email")).sendKeys("customer@test.com");
    driver.findElement(By.id("password")).sendKeys("wrongpass");
    driver.findElement(By.id("login-button")).click();

    // Assert error message is displayed
    WebElement error = driver.findElement(By.cssSelector(".error-message"));
    assertEquals("Invalid email or password.", error.getText());
}
}
```

Test execution flow:

1. **@BeforeAll** downloads the driver once.
 2. **@BeforeEach** creates a fresh ChromeDriver instance, maximizing the window and setting an implicit wait.
 3. **@Test** methods perform actions and assertions.
 4. **@AfterEach** quits the browser to ensure a clean state for the next test.
-